

Year 2 Statistics and scientific methods

Workflow: Projects, Pipes and functions

Erola Fenollosa

Aims of the session – using R for designing reproducible analytical workflows



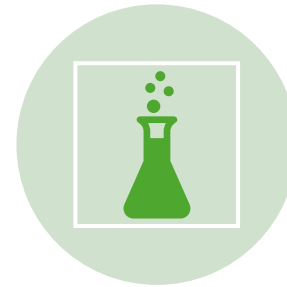
You already know dplyr and ggplot2.



Why projects matter



Why pipes change how we think



Why functions are essential

*Today you stop being R users. You start being **analysis designers***

Why using R Projects?

If I send you my script but not my folder structure, can you reproduce my work?

2025

The SORTEE Guidelines for Data and Code Quality Control in Ecology and Evolutionary Biology

Joel L. Pick^{1*}, Bethany J. Allen², Benedicte Bachelot³, Kevin R. Bairos-Novak⁴, Jack A. Brand^{5,6}, Barbara Class⁷, Tad Dallas⁸, Pietro B. D'Amelio⁹, Erola Fenollosa¹⁰, Esteban Fernández-Juricic¹¹, Dylan G. E. Gomes¹², Matthew J. Grainger¹³, Thomas Guillemaud¹⁴, Christian John¹⁵, Ruby Krasnow¹⁶, Malgorzata Lagisz^{17,18}, Sebastian Lequime¹⁹, Daniel S. Maynard²⁰, Shinichi Nakagawa¹⁸, Rose E. O'Dea²¹, Matthieu Paquet²², Quentin Petitjean²³, Alfredo Sánchez-Tójar²⁴, Natalie E. van Dis^{25,26}, Laura A. B. Wilson^{17,27}, Edward R. Ivimey Cook^{28*}

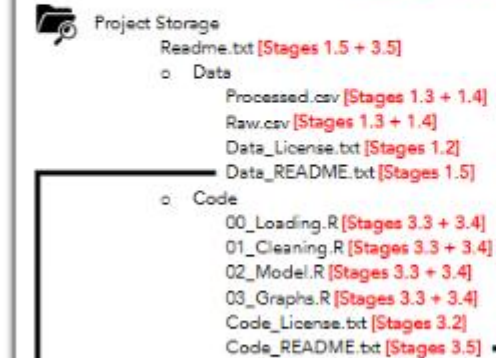
Project Structure

Title: Age and habitat influence the weight of offspring at hatching in *Nicrophorus vespilloides*

Contributors: Edward Ivimey-Cook

Date Created: 2024-10-21

Persistent Identifier: DOI/10.12345.6789 [Stages 1.1 + 3.1]



[Stage 1.5] Data_README.txt

Authors: Ed Ivimey-Cook
Email: Ed@Ivimey-Cook.com
Title: Age and habitat influence the weight of offspring at hatching in *Nicrophorus vespilloides*
Funders: SORTEE
Data License: CC-BY 4.0 in Data_License.txt
Summary: Data collected in May-June 2024 in Blackford Hill, Edinburgh on age, habitat, and weight of offspring produced across separate breeding attempts.

Raw.csv
ID: Individual identity of female (1-58)
Age: Age of individual sampled during breeding attempt (14, 30, or 60 days)
Habitat: Habitat that individuals were sampled (A = Grassland, B = Forest, C = Swamp)
Offspring_Weight_at_Hatch: Weight of offspring produced /mg

Processed.csv
ID: Individual identity of female (1-53)
Age_days: Age of individual sampled during breeding attempt (14, 30, or 60 days)
Habitat: Habitat that individuals were sampled (A = Grassland, B = Forest, C = Swamp)
Offspring_weight_g: Mean weight of offspring produced /mg

[Stage 3.5] Code_README.txt

Authors: Ed Ivimey-Cook
Email: Ed@Ivimey-Cook.com
Title: Age and habitat influence the weight of offspring at hatching in *Nicrophorus vespilloides*
Funders: SORTEE
Code License: MIT License in Code_License.txt

Code
00_Loading.R: Loading of essential functions and packages. Sourced in 01-03.
01_Cleaning.R: Imports Raw.csv - Cleaning functions to produce Processed.csv.
02_Model.R: Imports Processed.csv - Run the model on mean offspring weight at hatching against age and habitat.
03_Graphing.R: Imports Processed.csv - Ggplot2 graphs to produce Figure 1 of offspring weight at hatching.

Software and (Packages)

R version v4.3.1
{ggplot2} v3.5.1
{lme4} v1.1-35.5
{emmeans} v1.10.2
{dplyr} v1.1.4
{lmerTest} v3.1-3

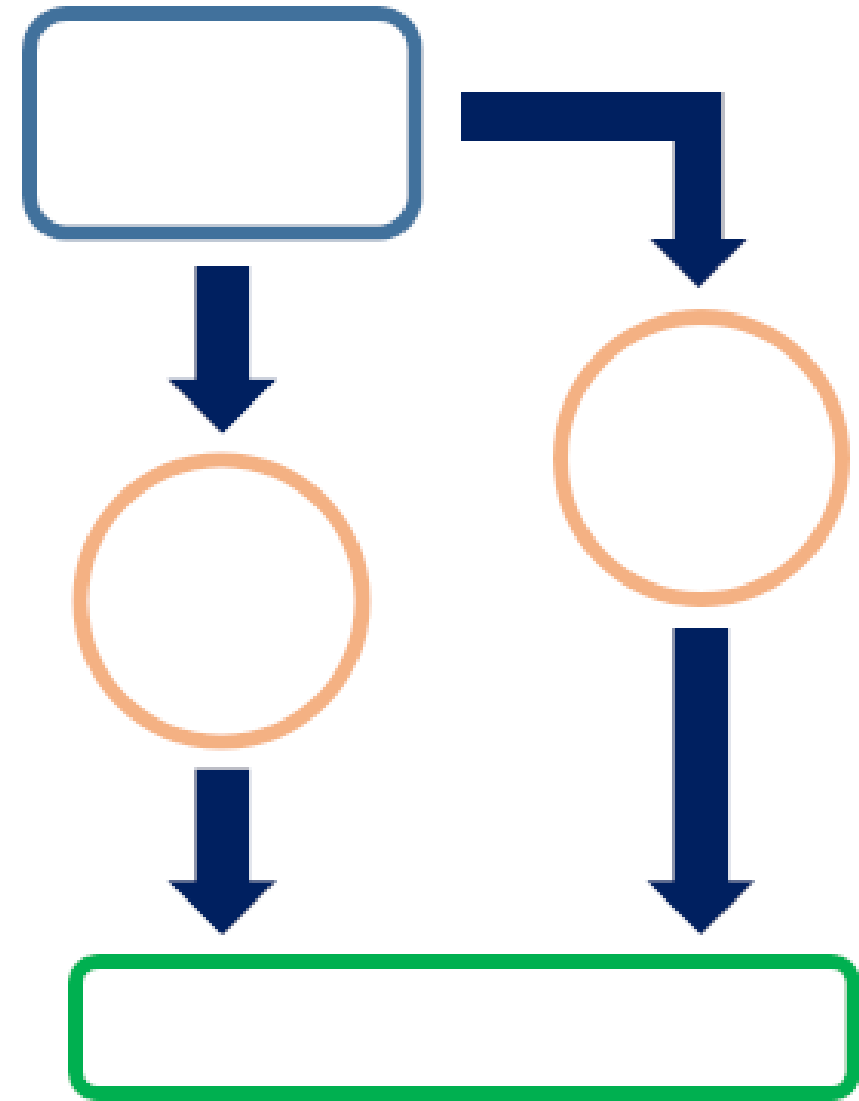
What is a Workflow?

A structured, reproducible path from raw data to result.

Have you ever heard of the concept **pseudocode**?

*Pseudocode is an informal, high-level description of an algorithm that uses a mix of plain English and structured, programming-like syntax. Pseudocode is the **intermediate state between an idea and its implementation(code)** in a high-level language.*

In today's world, understanding how to structure solutions in pseudocode can be more valuable than writing the final code itself.



Essential elements: Pipes

Pipes describe a sequence of transformations (It is just another way of doing things)

Pipes are about:

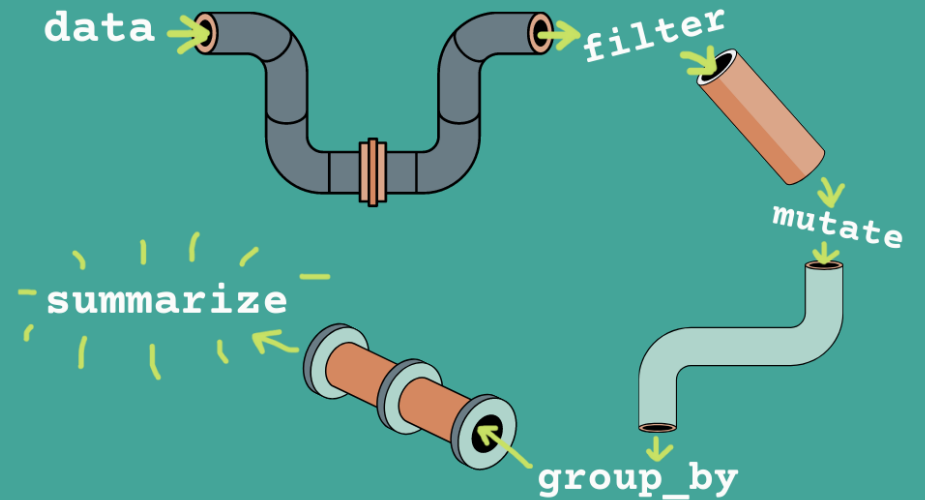
- Readability
- Sequential logic
- Cognitive load reduction

Which one tells the story better?

```
x1 <- filter(data, ...)  
x2 <- mutate(x1, ...)  
x3 <- summarise(x2, ...)
```

```
data %>%  
  filter(...) %>%  
  mutate(...) %>%  
  summarise(...)
```

Using pipes in 



%>%

This is a pipe.

~~*Ceci n'est pas une pipe.*~~

Pipes and functions

If you repeat the same analysis for 20 species, will you copy and paste code 20 times?



```
filtered_data <- data[data$Species == "Bufo bufo", ]  
result <- mean(filtered_data$Mass)  
result
```



```
filtered_data <- filter(data, Species == "Bufo bufo")  
result <- summarise(filtered_data, mean_mass = mean(Mass))  
result
```



```
data %>% filter(Species == "Bufo bufo") %>% summarise(mean_mass = mean(Mass))
```



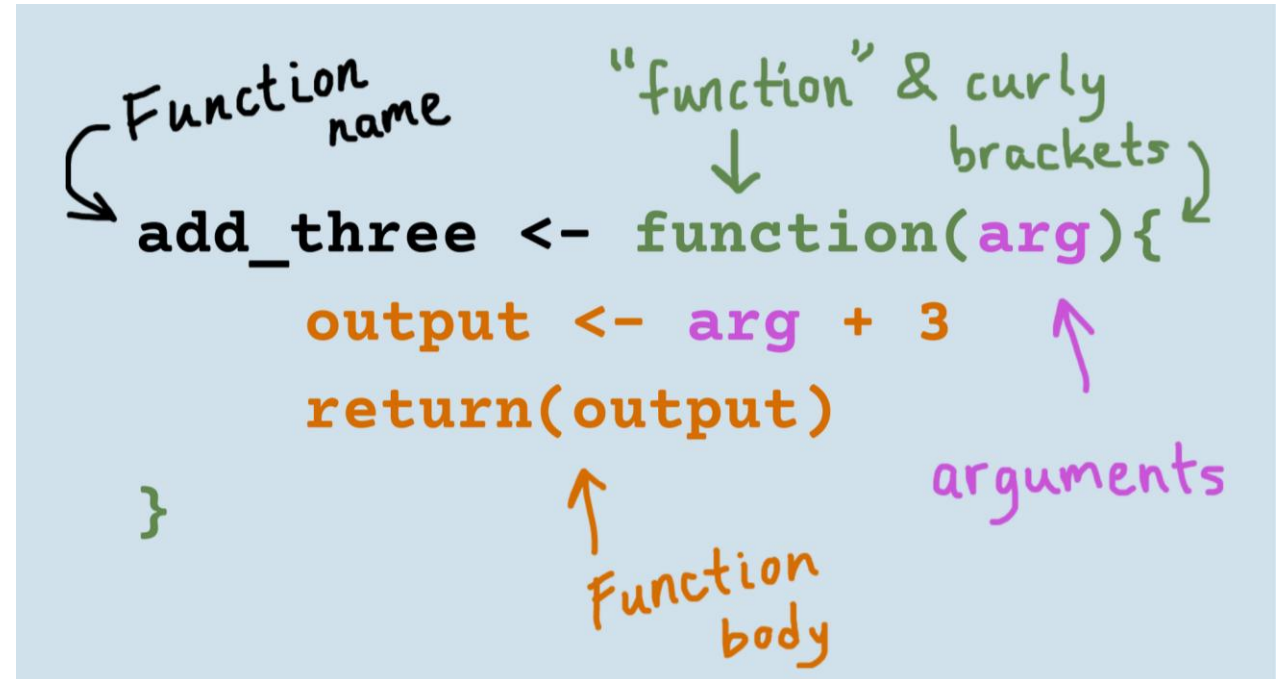
```
analyse_species <- function(data, species_name) {  
  data %>%  
    filter(Species == species_name) %>%  
    summarise(mean_mass = mean(Mass))  
}
```


Elements of a FUNction

Takes inputs (arguments)

Performs operations

Returns output



The diagram shows the R function syntax: `add_three <- function(arg){`
 `output <- arg + 3`
 `return(output)`
`}`
Annotations include:
- "Function name" pointing to `add_three`
- "function" & curly brackets pointing to `function(arg){`
- "arguments" pointing to `arg`
- "Function body" pointing to the lines inside the curly brackets

EXAMPLE 1

DEFINE THE FUNCTION

```
generate_random <- function(x) {  
  runif(x)  
}
```

USE THE FUNCTION

```
generate_random(5)
```

EXAMPLE 2

DEFINE THE FUNCTION

```
pick_random_person <- function(people) {  
  sample(people, size = 1)  
}
```

USE THE FUNCTION

```
students <- c("Ana", "Leo", "Marta", "James", "Sofia")  
pick_random_person(students)
```

<https://www.rforecology.com/post/how-to-create-your-own-function-in-r/>

Good practices: Testing, Error Handling, and Documenting

- Use descriptive names
- Document Your Functions
- Validate Inputs
- Use Default Argument Values
- Test Your Functions

1. What do you think this function called `b()` is doing?

```
b <- function(x, y) {  
  if (format(Sys.Date(), "%m-%d") == paste(x, y, sep = "-")) {  
    print("congratulations!!!!!!")  
  }  
}
```

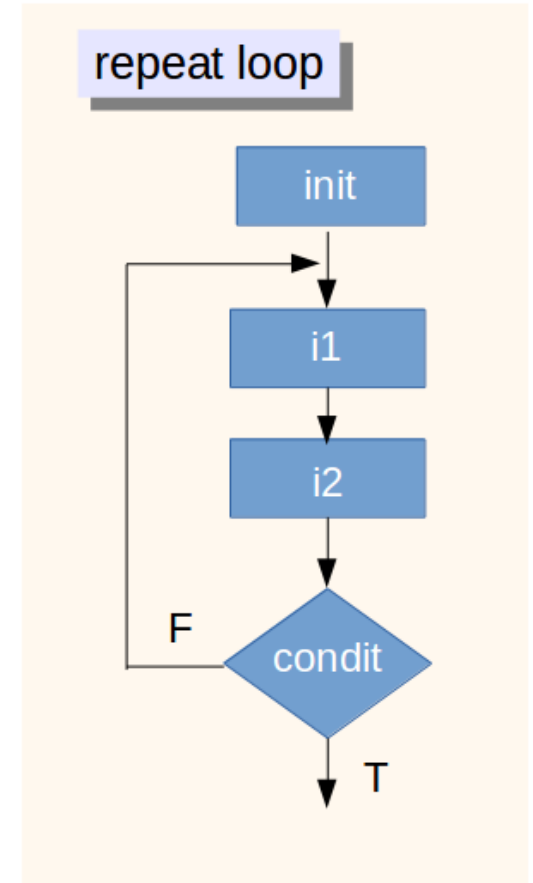
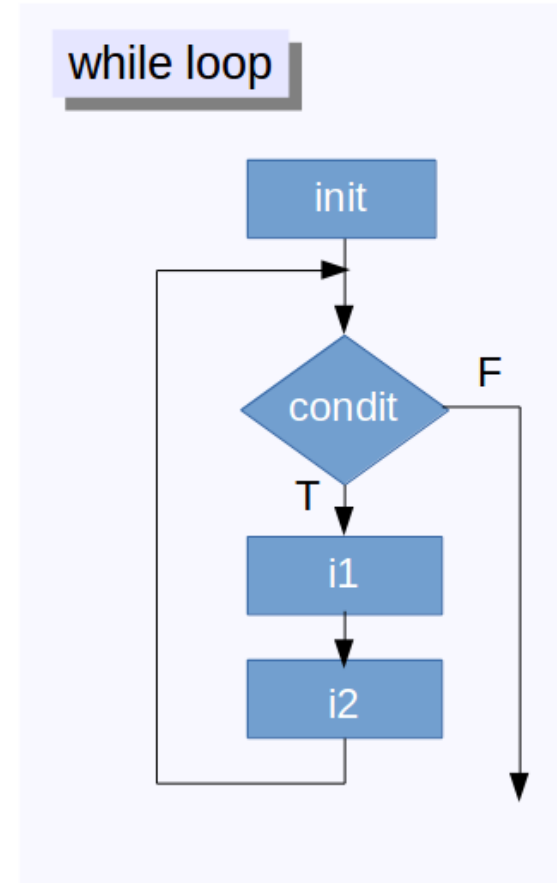
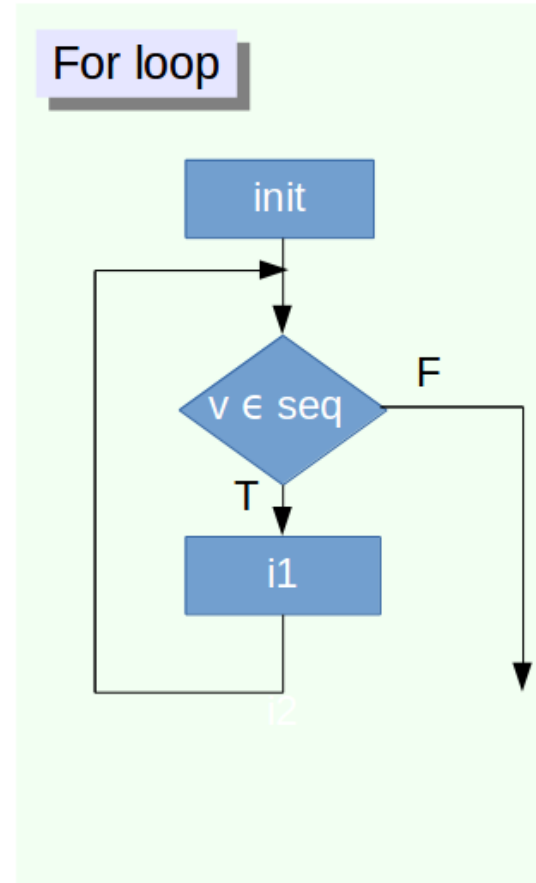
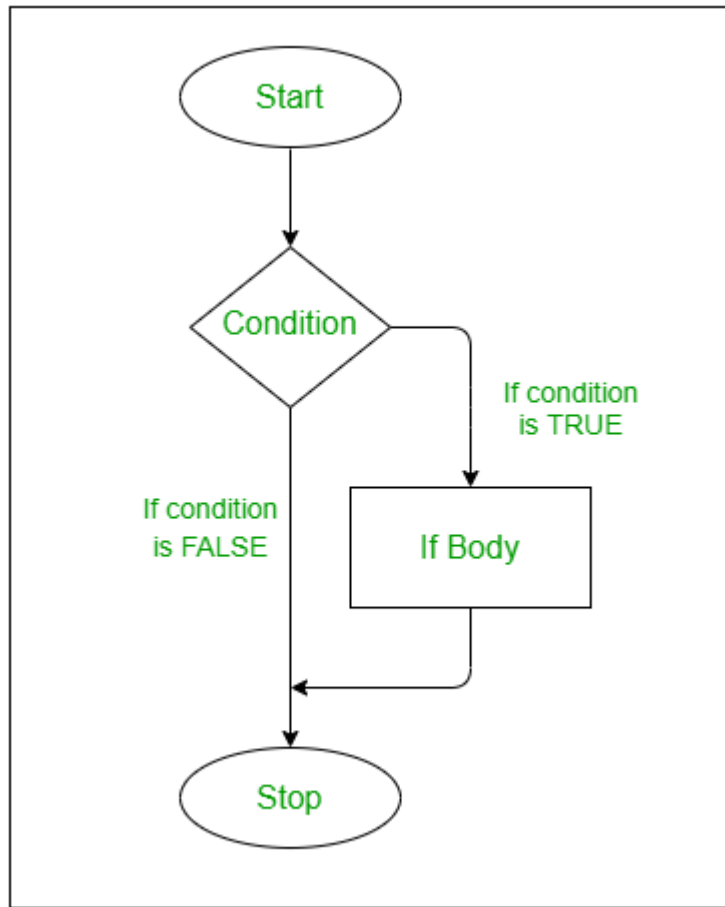
2. A proper function (perhaps too much...)

```
#' Check whether Today Is a Birthday  
#'  
#' This function checks whether the provided birth month and day  
#' match today's date. If it is the user's birthday, it returns  
#' a congratulatory message. Otherwise, it returns a neutral message.  
#'  
#' @param birth_month Numeric. Month of birth (1-12).  
#' @param birth_day Numeric. Day of birth (1-31).  
#' @param current_date Date. The date to compare against.  
#' Defaults to Sys.Date().  
#'  
#' @return Character string indicating whether today is the birthday.  
check_birthday <- function(birth_month,  
                           birth_day,  
                           current_date = Sys.Date()) {  
  
  # ---- Input validation ----  
  
  if (!is.numeric(birth_month) || !is.numeric(birth_day)) {  
    stop("birth_month and birth_day must be numeric.")  
  }  
  
  if (birth_month < 1 || birth_month > 12) {  
    stop("birth_month must be between 1 and 12.")  
  }  
  
  if (birth_day < 1 || birth_day > 31) {  
    stop("birth_day must be between 1 and 31.")  
  }  
  
  if (!inherits(current_date, "Date")) {  
    stop("current_date must be of class Date.")  
  }  
  
  # ---- Extract current month and day ----  
  
  current_month <- as.numeric(format(current_date, "%m"))  
  current_day <- as.numeric(format(current_date, "%d"))  
  
  # ---- Check birthday ----  
  
  if (birth_month == current_month && birth_day == current_day) {  
    return("Congratulations!!!!!!")  
  } else {  
    return("Today is not your birthday.")  
  }  
}
```




Functions + control flow

- Choices
- Loops



Custom plot function

Let's get back to earth... and explore a function that I use to use a lot. First, identify the elements.

DEFINE THE FUNCTION

```
plot_countries_trend <- function(df, countries, var_name, smooth = TRUE, add_points = TRUE, line_size = 1, point_size = 2,
                                palette = NULL, title = NULL) {
  # ---- Defensive checks ----
  if (!is.data.frame(df)) stop("df must be a data frame.")
  if (!"year" %in% names(df)) stop("df must contain a 'year' column.")
  if (!is.character(var_name) || length(var_name) != 1) stop("var_name must be a single string.")
  if (!var_name %in% names(df)) stop(paste0("Variable '", var_name, "' not found in df."))
  if (missing(countries) || length(countries) == 0) stop("Please supply one or more country names in 'countries'.")
  if (!all(countries %in% df[["country"]])) { missing_c <- setdiff(countries, unique(df[["country"]]))
    stop(paste0("These countries not found in df: ", paste(missing_c, collapse = ", "))) }
  if (!is.logical(smooth) || length(smooth) != 1) stop("'smooth' must be TRUE or FALSE.")
  if (!is.logical(add_points) || length(add_points) != 1) stop("'add_points' must be TRUE or FALSE.")

  # ---- Prepare data ----
  plot_data <- df %>% dplyr::filter(country %in% countries) %>% dplyr::select(year, country, value = .data[[var_name]]) %>%
    dplyr::arrange(country, year)

  # ---- Build the plot ----
  p <- ggplot(plot_data, aes(x = year, y = value, color = country, group = country)) +
    geom_line(size = line_size)

  if (isTRUE(add_points)) { p <- p + geom_point(size = point_size) }
  if (isTRUE(smooth)) { p <- p + geom_smooth(se = TRUE, method = "loess", aes(fill = country), alpha = 0.15, show.legend = FALSE) }

  # Color palette
  if (!is.null(palette)) {
    if (length(palette) < length(unique(plot_data$country))) {
      warning("Palette shorter than number of countries; colors will recycle.")
    }
    p <- p + scale_color_manual(values = palette)
  }

  # Labels and theme
  if (is.null(title)) {
    title <- paste0("Trend of ", var_name, " for selected countries")
  }

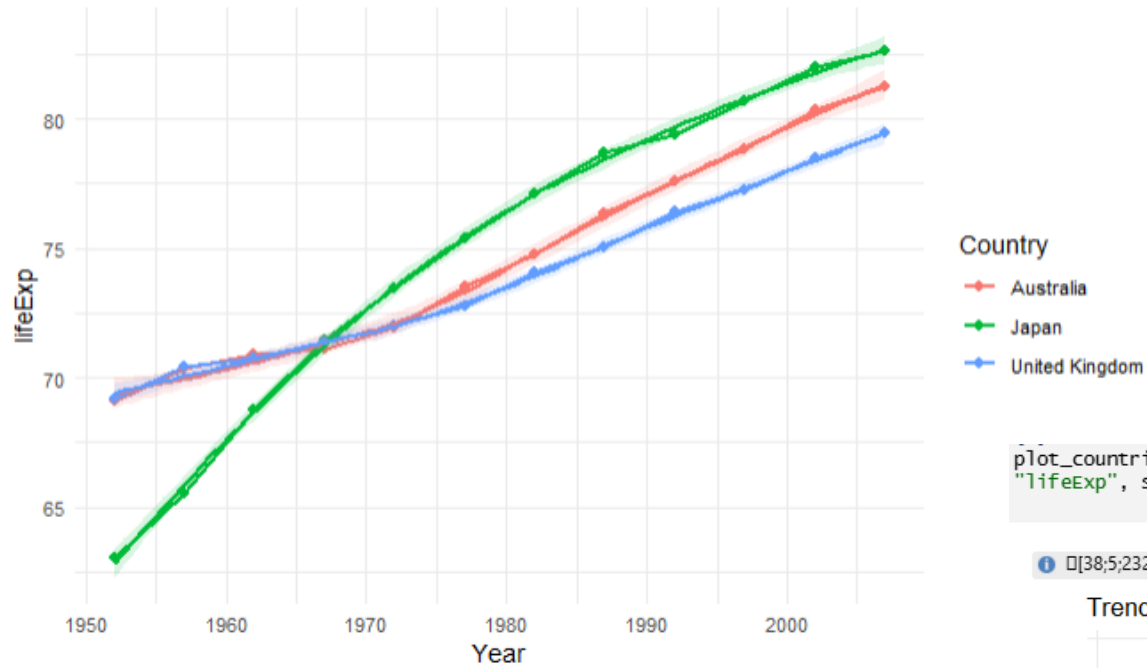
  p <- p + labs(x = "Year", y = var_name, color = "Country", title = title) +
    theme_minimal() + theme(legend.position = "right")
  p
}
```

```
{r}
plot_countries_trend(gapminder, countries = c("Japan", "United Kingdom", "Australia"), var_name = "lifeExp", smooth = TRUE)
```

USE THE FUNCTION

□[38;5;232m`geom\_smooth()` using formula = 'y ~ x'□[39m

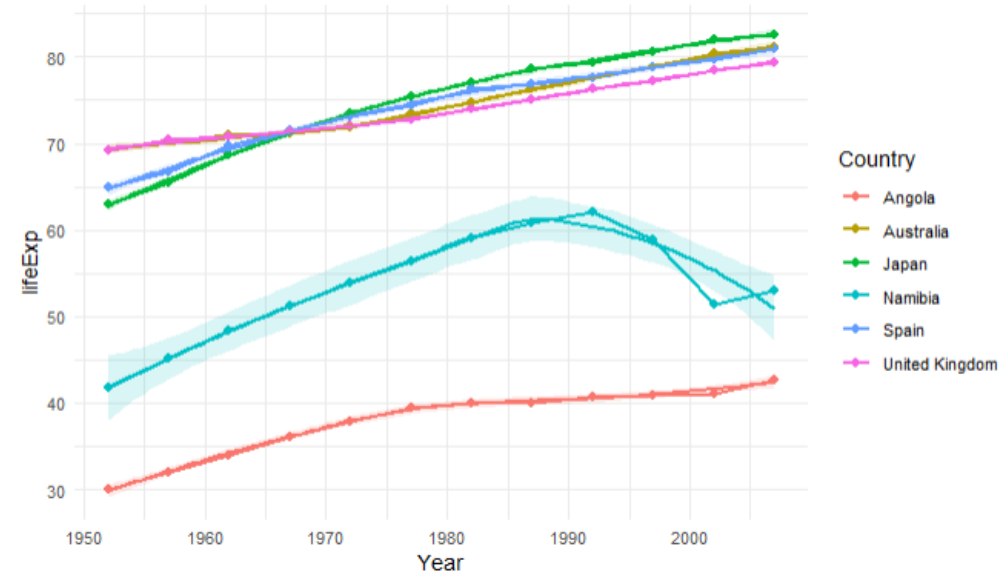
Trend of lifeExp for selected countries



```
plot_countries_trend(gapminder, countries = c("Japan", "United Kingdom", "Australia", "Spain", "Angola", "Namibia"), var_name = "lifeExp", smooth = TRUE)
```

□[38;5;232m`geom\_smooth()` using formula = 'y ~ x'□[39m

Trend of lifeExp for selected countries



Your turn: The *gapminder* dataset

[https://www.gapminder.org/tools/#\\$chart-type=bubbles&url=v2](https://www.gapminder.org/tools/#$chart-type=bubbles&url=v2)

```
> head(gapminder)
```

```
# A tibble: 6 x 6
```

	country	continent	year	lifeExp	pop	gdpPer cap
	<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
1	Afghanistan	Asia	1952	28.8	8425333	779.
2	Afghanistan	Asia	1957	30.3	9240934	821.
3	Afghanistan	Asia	1962	32.0	10267083	853.
4	Afghanistan	Asia	1967	34.0	11537966	836.
5	Afghanistan	Asia	1972	36.1	13079460	740.
6	Afghanistan	Asia	1977	38.4	14880372	786.

